

Integrator System Architecture

Contents

1. What the Theory Requires of an Integrator	1
2. Integrator Internal Architecture	3
3. Data Flow per Mechanism Epoch	7
4. Requirement-to-Subsystem Traceability	8
5. Major Implementation Challenges	10
6. Challenge Summary	14
7. Key Architectural Insight	15

Detailed internal architecture for the integrator component, synthesised from the requirements of all three IEEE TSC papers.

1. What the Theory Requires of an Integrator

The integrator sits at the boundary between service composition (the physical sub-DAG of compute resources) and market design (the abstract slice traded at auction). Every theoretical result in the trilogy constrains what happens inside that boundary.

1.1 Requirements from Paper 1 (Base Framework)

ID	Requirement	Source	Type
R1.1	Encapsulate a service-dependency sub-DAG into a polymatroidal slice with scalar capacity equal to the max-flow of the sub-DAG	Prop. 1, Lemma 1	Hard
R1.2	Preserve gross-substitutes (GS) structure at the market interface (unit demand, additive separability, fixed attributes within epoch)	Lemma 1, GS1-GS3	Hard
R1.3	Enforce governance capacity bounds $x_l \leq u_l(t)$ per leaf, intersected with resource feasibility before each epoch	Prop. 1 extension	Hard

ID	Requirement	Source	Type
R1.4	Support cross-domain coordination : an integrator may procure resources from multiple local marketplaces and aggregate them into a single slice type	Architecture (Sec. III)	Hard
R1.5	Maintain a rank function $f(S)$ for the polymatroid that is consistent with the physical sub-DAG's max-flow	Prop. 1 proof	Hard
R1.6	Operate within the two-level hybrid architecture : Level 1 (slice marketplace) above, Level 2 (local resource marketplaces) below	Architecture (Sec. III)	Hard

1.2 Requirements from Paper 2 (Credibility)

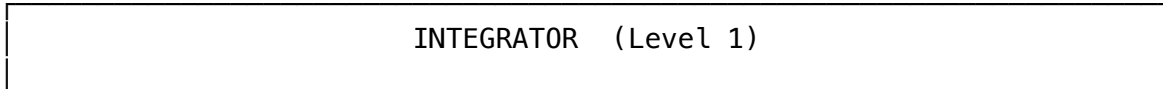
ID	Requirement	Source	Type
R2.1	Execute the prescribed mechanism faithfully (allocation and payments match the announced rules)	Credibility (Def. 2)	Hard
R2.2	Provide a commitment device so agents can verify execution: broadcast channel (ascending auction), smart contract (DRA), or neutral exchange	Thm. 2, B1-B2	Hard
R2.3	Accept domain separation (the operator entity is structurally distinct from agents and resource providers)	T3	Hard
R2.4	Revenue model: either payment-based $\pi_j = \sum_i p_i^{(j)} - \text{cost}_j$ (requires commitment) or per-unit fee ϕ (VCG-credible without commitment)	T3, Thm. 2	Design choice
R2.5	Produce an append-only transcript of $(bids, allocation, payments)$ per epoch for external audit	Thm. 2	Hard
R2.6	When using ascending auction: support broadcast of agent demand/exit messages so clinch quantities are publicly reconstructible	Thm. 2, B1-B2	Conditional

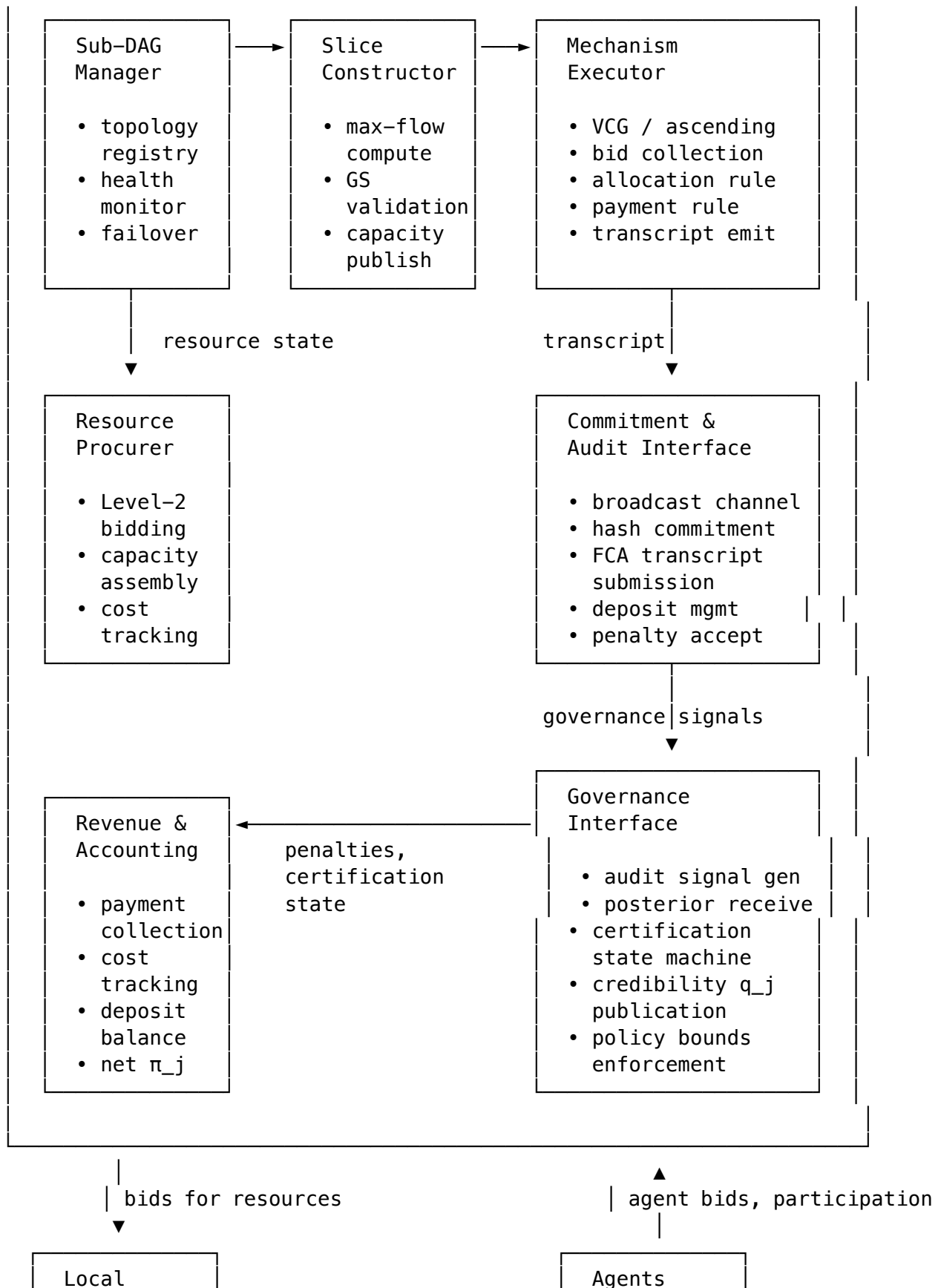
1.3 Requirements from Paper 3 (Adversarial Credibility)

ID	Requirement	Source	Type
R3.1	Never share private bid information $\{b_i^{(j)}\}$ with other integrators outside the audit protocol	Thm. 1	Hard
R3.2	Participate in cross-market audit : submit encrypted bid transcripts to FCA, or commit hash $H_j = \text{Hash}(bids_j, alloc_j, payments_j, \{H_k\})$ including adjacent integrators' hashes	Cor. 1, Rem. 1	Hard
R3.3	Post a slashing deposit $B \geq \bar{\Delta} / \Pr(\text{detected} \mid \delta)$ at registration, subject to partial or full forfeiture on detected deviation	Thm. 2	Hard
R3.4	When facing contextual-bandit adversaries: accept state-dependent penalty schedule $p_F(s) = \alpha \cdot \bar{\Delta}(s)$ for all observable states s	Thm. 2(ii)	Conditional
R3.5	Generate audit signals $z_t \in \{\text{compliant}, \text{anomalous}\}$ with known emission probabilities for Bayesian type tracking	Prop. 1	Hard
R3.6	Accept decertification when posterior $\hat{\pi}_t^S > \tau_d$, with recertification possible under elevated prior	Prop. 1(ii)	Hard
R3.7	Accept forgetting-factor updates ρ tuned by the marketplace for non-stationary type tracking	Prop. 1(iii)	Hard
R3.8	Publish credibility quality q_j derived from the posterior, enabling strategic agents to make informed participation decisions	Prop. 2	Hard
R3.9	Coalition penalty must exceed per-member share: $B \geq \sum_{j \in C} \pi_j(\delta_C) / C $	Thm. 1	Hard

2. Integrator Internal Architecture

The requirements above decompose naturally into seven internal subsystems. Each subsystem maps to one or more theoretical requirements.





Marketplace
(Level 2)

(Level 0)

2.1 Sub-DAG Manager

Owns: The physical service-dependency graph $G_j = (V_j, E_j)$ with per-node capacities.

Responsibilities: - Maintain a registry of compute nodes (device, edge, cloud) and the dependency edges between them. - Continuously monitor node health, latency, and available capacity. - Detect topology changes (node failure, link degradation, new resource availability) and propagate them to the Slice Constructor. - Enforce that the sub-DAG remains tree or series-parallel after any topology change (R1.1). If a reconfiguration would introduce entanglement, reject it or decompose into multiple sub-DAGs.

Interfaces: - IN: heartbeats and capacity reports from compute nodes; topology-change events. - OUT: current G_j with per-node capacities to Slice Constructor; resource requirements to Resource Procurer.

Maps to: R1.1, R1.4, R1.6.

2.2 Slice Constructor

Owns: The abstraction from sub-DAG to tradeable slice.

Responsibilities: - Compute the **max-flow** of G_j to determine aggregate slice capacity $f(\{l\})$ for each leaf service l . This is the scalar capacity offered at the Level-1 marketplace. - Validate that the resulting slice satisfies **GS conditions** (R1.2): unit demand per agent per task, additive separability across tasks, fixed attributes within epoch. - Intersect resource capacity $\mathcal{X}_{\text{res}}$ with governance bounds $\mathcal{X}_{\text{gov}}$ to produce the feasible polymatroid for the upcoming epoch. - Publish the slice catalogue (available types, capacities, attributes) to the Mechanism Executor.

Critical invariant: The rank function $f(S) = \text{max-flow}(G_j, S)$ must be **submodular** (R1.5). If the sub-DAG violates this (e.g., due to entangled topology), the Slice Constructor must either: - Conservatively underestimate capacity (safe but wasteful), or - Signal to Sub-DAG Manager to decompose the topology.

Interfaces: - IN: current G_j and capacities from Sub-DAG Manager; governance bounds from Governance Interface. - OUT: polymatroid parameters (E, f) and slice catalogue to Mechanism Executor.

Maps to: R1.1, R1.2, R1.3, R1.5.

2.3 Mechanism Executor

Owns: The auction/mechanism that allocates slices to agents.

Responsibilities: - Collect bids b_i from agents (or demand schedules in ascending format). - Compute the welfare-maximising allocation x^* over the polymatroid. - Compute payments p_i via VCG or clinching rule. - Emit an **append-only transcript** of $(bids, allocation, payments)$ (R2.5). - If ascending auction mode: manage the price-clock, broadcast agent demand/exit messages (R2.6), compute clinch quantities.

Mechanism modes (selectable per deployment): 1. **Sealed-bid VCG**: Simpler. Agents submit sealed bids; integrator computes allocation and VCG payments. Vulnerable to execution deviation unless externally audited. 2. **Ascending with broadcast**: Agents broadcast their own messages. Clinch quantities are publicly reconstructible. Embeds commitment into the mechanism itself. 3. **Deferred-revelation auction (DRA)**: Agents submit hash-committed bids; revelation and allocation happen in a second phase. Requires blockchain or escrow for commitment.

Critical property: The Mechanism Executor must faithfully implement the announced mechanism (R2.1). Any deviation (bid suppression, payment inflation, allocation manipulation) must be detectable via the transcript.

Interfaces: - IN: polymatroid parameters from Slice Constructor; agent bids from agents. - OUT: allocation x^* and payments p_i to agents and Revenue module; transcript to Commitment & Audit Interface.

Maps to: R1.2, R2.1, R2.2, R2.5, R2.6.

2.4 Resource Procurer

Owns: The integrator's demand-side participation in Level-2 local marketplaces.

Responsibilities: - Translate the committed slice allocation into concrete resource bids at one or more local marketplaces (R1.4, R1.6). - Manage multi-marketplace procurement: if a slice requires resources from domain A (edge) and domain B (cloud), bid in both markets and assemble the results. - Track procurement costs for the Revenue module. - Handle procurement failures: if a local marketplace cannot supply the required resources, either degrade the slice (reduce capacity) or procure from an alternative source.

Interfaces: - IN: required resources from Sub-DAG Manager; committed allocations from Mechanism Executor. - OUT: resource bids to Level-2 marketplaces; procurement costs to Revenue module; actual capacity to Sub-DAG Manager.

Maps to: R1.4, R1.6.

2.5 Commitment and Audit Interface

Owns: All external verifiability and audit interactions.

Responsibilities: - **Commitment device management**: Operate the broadcast channel (ascending mode), hash-commitment protocol (DRA mode), or transcript escrow (institutional mode). - **Cross-market hash chain**: At epoch end, compute $H_j = \text{Hash}(bids_j, alloc_j, payments_j, \{H_k : k \in \text{adj}(j)\})$, incorporating hashes from adjacent integrators (R3.2). - **FCA transcript submission**: Encrypt and submit the epoch transcript to the Federated Certification Authority for cross-market consistency verification (R3.2). - **Deposit management**: Maintain the slashing deposit B (R3.3), handle partial forfeiture on penalty events. - **Penalty acceptance**: Receive and execute penalty decisions from the FCA/Penalty Engine: state-independent slashing or state-dependent fines (R3.3, R3.4).

Critical constraint: This module must be **tamper-evident**. The transcript it produces must be the same one seen by agents (via broadcast) and by the FCA (via submission). Any divergence is itself a detectable deviation.

Interfaces: - IN: transcript from Mechanism Executor; hashes from adjacent integrators; penalty decisions from Governance Interface. - OUT: committed transcript to FCA; broadcast messages to agents; deposit status to Revenue module.

Maps to: R2.2, R2.5, R3.1, R3.2, R3.3, R3.4, R3.9.

2.6 Governance Interface

Owns: All interactions with the external governance stack (FCA, Reputation Service, Policy Engine, Market Segmentation Controller).

Responsibilities: - **Audit signal generation:** Produce signal $z_t \in \{\text{compliant, anomalous}\}$ from internal consistency checks (R3.5). The emission probabilities $\Pr(z_t \mid \omega_t)$ must match the governance model's assumptions. - **Posterior reception:** Receive the current Bayesian posterior $\hat{\pi}_t^S$ from the FCA/Reputation Service. - **Certification state machine:** Track the integrator's own certification status (certified, warning, decertified, recertifying) and comply with state transitions (R3.6). - **Credibility publication:** Compute and publish q_j (credibility quality) derived from the posterior, for agent consumption (R3.8). - **Policy bounds enforcement:** Receive per-leaf governance bounds $u_l(t)$ from the Policy Engine and forward them to the Slice Constructor (R1.3). - **Forgetting factor acceptance:** Receive and apply the marketplace-tuned ρ for non-stationary type tracking (R3.7).

Interfaces: - IN: posterior $\hat{\pi}_t^S$ and certification decisions from FCA; governance bounds from Policy Engine; forgetting factor from marketplace. - OUT: audit signals to FCA; credibility quality q_j to agents and Market Segmentation Controller; governance bounds to Slice Constructor; certification state to Revenue module.

Maps to: R1.3, R3.5, R3.6, R3.7, R3.8.

2.7 Revenue and Accounting

Owns: Financial tracking and deposit management.

Responsibilities: - Collect payments p_i from agents (via Mechanism Executor outcomes). - Track procurement costs from Resource Procurer. - Compute net revenue $\pi_j = \sum_i p_i^{(j)} - \text{cost}_j$ (R2.4). - Maintain deposit balance B_t , accounting for any slashing events. - Provide financial data for penalty calibration (historical Δ estimation).

Interfaces: - IN: payments from Mechanism Executor; costs from Resource Procurer; penalty events from Commitment & Audit Interface; certification state from Governance Interface. - OUT: net revenue reports; deposit balance; historical revenue data for penalty calibration.

Maps to: R2.4, R3.3.

3. Data Flow per Mechanism Epoch

A single epoch proceeds in seven phases:

Phase 1: Pre-epoch preparation

Sub-DAG Manager $\rightarrow$ Slice Constructor: current G_j , capacities

Governance Interface $\rightarrow$ Slice Constructor: governance bounds $u_l(t)$

Slice Constructor: compute max-flow, validate GS, produce (E, f)

Phase 2: Mechanism execution

Agents → Mechanism Executor: bids b_i (or demand schedules)

Mechanism Executor: compute x^* , p_i

Mechanism Executor → Commitment & Audit: transcript (bids, x^* , p)

Phase 3: Commitment

Commitment & Audit: broadcast/hash-commit transcript

Commitment & Audit ↔ Adjacent integrators: exchange hashes H_k

Commitment & Audit → FCA: submit encrypted transcript

Phase 4: Resource procurement

Mechanism Executor → Resource Procurer: required resources for x^*

Resource Procurer → Level-2 marketplaces: resource bids

Resource Procurer → Revenue: procurement costs

Phase 5: Settlement

Revenue: collect p_i from agents, pay Level-2 costs

Revenue: compute $\pi_j = \sum p_i - \text{costs}$

Phase 6: Audit and governance

FCA: cross-market consistency check

FCA → Governance Interface: audit signal z_t , updated $\hat{\pi}_t^S$

Governance Interface: update certification state

Governance Interface → agents: publish q_j

Phase 7: Penalty (if triggered)

FCA → Commitment & Audit: penalty decision

Commitment & Audit → Revenue: execute slashing

Revenue: update deposit balance B

4. Requirement-to-Subsystem Traceability

Requirement	Sub-DAG	Slice Constr.	Mech. Exec.	Res. Procurer	Commit & Audit	Governance	Revenue
R1.1 Poly-matroid encapsulation	•	•					
R1.2 GS preservation		•	•				
R1.3 Governance bounds		•				•	

Requirement	Sub-DAG	Slice Constr.	Mech. Exec.	Res. Procurer	Commit & Audit	Governance	Revenue
R1.4 Cross-domain procurement	•			•			
R1.5 Sub-modular rank function		•					
R1.6 Two-level architecture	•			•			
R2.1 Faithful execution			•				
R2.2 Commitment device			•		•		
R2.4 Revenue model							•
R2.5 Append-only transcript			•		•		
R2.6 Broadcast messages			•		•		
R3.1 No bid sharing					•		
R3.2 Cross-market audit					•	•	
R3.3 Slashing deposit					•		•
R3.4 State-dependent penalties					•		•

Requirement	Sub-DAG	Slice Constr.	Mech. Exec.	Res. Procurer	Commit & Audit	Governance	Revenue
R3.5 Audit signal genera- tion						•	
R3.6 Decertifi- cation accep- tance						•	
R3.7 Forgetting factor						•	
R3.8 Credibility publica- tion						•	
R3.9 Coalition penalty bound					•		•

5. Major Implementation Challenges

Challenge 1: Max-Flow Recomputation Under Dynamic Topology

Problem. The Slice Constructor must recompute $f(S) = \text{max-flow}(G_j, S)$ whenever the sub-DAG changes (node failure, capacity fluctuation, new resource onboarding). For a sub-DAG with $|V|$ nodes and $|E|$ edges, max-flow is $O(|V| \cdot |E|)$ per subset S . The rank function must be evaluated for exponentially many subsets during mechanism execution.

Why it matters. The integrator advertises slice capacities at the start of each epoch. If the sub-DAG changes mid-epoch (e.g., an edge node goes offline), the advertised capacity may no longer be deliverable. This creates a gap between the polymatroid the market traded on and the physical resources available.

Tension. Recomputing more frequently improves accuracy but increases latency and may destabilise the mechanism (agents see changing capacities mid-auction). Recomputing less frequently risks over-promising.

Severity: HIGH. This is the fundamental abstraction that makes the entire market architecture work. If max-flow computation is too slow or too stale, the polymatroidal guarantee breaks.

Challenge 2: GS Validation at the Boundary

Problem. The Slice Constructor must verify that the slices it produces satisfy GS conditions (GS1: unit demand, GS2: additive separability, GS3: fixed attributes). These are properties of the

interaction between the slice definition and agent valuations, not of the slice alone.

Why it matters. GS is required for the welfare-maximising allocation to coincide with a Walrasian equilibrium (Gul-Stacchetti), which in turn is required for VCG to be DSIC (Paper 1, Prop. 2). If GS fails, the mechanism may not be truthful, and agents have incentive to misreport.

Tension. The integrator controls slice definition but not agent valuations. GS2 (no cross-task complementarities) is a restriction on agent preferences that the integrator cannot enforce; it can only design slices to make complementarities unlikely (e.g., by bundling complementary sub-services into a single slice type).

Severity: HIGH. The DSIC guarantee is the foundation of the entire incentive structure. If GS fails silently, all downstream guarantees (truthful bidding, efficient allocation, penalty calibration) are undermined.

Challenge 3: Transcript Integrity Across Mechanism Modes

Problem. The Commitment & Audit Interface must produce a transcript that is simultaneously: - **Complete:** contains all information needed for the FCA to verify bid-allocation-payment consistency. - **Tamper-evident:** any modification is detectable. - **Privacy-preserving:** does not leak agent bids to competing integrators or third parties beyond what the audit requires.

Why it matters. The entire credibility architecture (Papers 2-3) rests on the assumption that deviations are detectable from the transcript. If the integrator can produce a consistent-looking but fabricated transcript, all detection mechanisms fail.

Tension. In sealed-bid mode, only the integrator sees the bids; the transcript is self-reported. In ascending mode, agents broadcast their own messages, making fabrication harder but exposing bid information. In DRA mode, hash commitments provide some integrity but require a trusted execution environment or blockchain for verification.

Severity: CRITICAL. This is the single point of failure for the entire governance stack. A compromised transcript module renders the FCA, penalties, and type tracking useless.

Challenge 4: Deposit Sizing Under Uncertainty

Problem. The slashing deposit $B \geq \bar{\Delta} / \Pr(\text{detected} \mid \delta)$ requires estimating: - $\bar{\Delta}$: the maximum expected deviation gain, which depends on agent bid distributions, the non-modularity gap γ_{ij} , and the specific deviation strategy. - $\Pr(\text{detected} \mid \delta)$: the detection probability, which depends on audit frequency, signal accuracy, and the deviation's subtlety.

Why it matters. If B is set too low, deviation is profitable despite the deposit. If B is set too high, it creates a barrier to entry that reduces competition (fewer integrators can afford to participate), which ironically weakens the market-discipline layer.

Tension. $\bar{\Delta}$ is not directly observable; it must be estimated from historical data or worst-case analysis. In new markets with no history, conservative (high) deposits may be necessary, potentially excluding smaller integrators.

Severity: HIGH. Under-depositing creates a systematic incentive to deviate; over-depositing suppresses competition. Both undermine the market.

Challenge 5: State-Dependent Penalty Estimation

Problem. For contextual-bandit adversaries (Thm. 2(ii)), the penalty schedule $p_F(s) = \alpha \cdot \bar{\Delta}(s)$ must hold for ALL observable states $s \in \mathcal{S}$. This requires: - Defining the state space $\mathcal{S}$ (what market conditions are observable?). - Estimating $\bar{\Delta}(s)$ for each state (how profitable is deviation in state s ?). - Ensuring no state is left under-penalised.

Why it matters. If even a single state s^* has $p_F(s^*) < \bar{\Delta}(s^*)$, a contextual learner will discover and repeatedly exploit it, converging to deviation frequency $\pi(s^*)$ (the stationary probability of that state).

Tension. The state space may be high-dimensional (load level, number of active agents, bid distribution, time of day, etc.). Estimating $\bar{\Delta}(s)$ requires sufficient historical data per state, which grows exponentially with state dimensionality. Coarsening the state space (fewer states) makes estimation feasible but may miss exploitable substates.

Severity: HIGH. This is the gap between Theorem 2(i) (fixed penalties work for simple learners) and Theorem 2(ii) (contextual learners need state-dependent penalties). Bridging this gap in practice is an open estimation problem.

Challenge 6: Audit Signal Calibration

Problem. The Governance Interface must generate audit signals z_t with known emission probabilities: $\Pr(\text{anomalous} \mid \text{honest}) = \varepsilon$ and $\Pr(\text{anomalous} \mid \text{strategic}) = 1 - \varepsilon$, where $\varepsilon < 1/2$. The Bayesian type tracker (Prop. 1) assumes these probabilities are fixed and known.

Why it matters. If the actual false-positive rate differs from the assumed ε , the posterior $\hat{\pi}_t^S$ is miscalibrated. Too many false positives cause premature decertification of honest integrators; too few cause delayed detection of strategic ones.

Tension. In practice, the “anomalous” signal is generated by statistical tests on the transcript (bid-allocation consistency, payment inflation tests). The false-positive rate of these tests depends on the bid distribution, which changes over time. A fixed ε is a modelling convenience, not a practical guarantee.

Severity: MEDIUM-HIGH. Miscalibrated signals degrade the entire dynamic credibility system. The theory provides convergence guarantees conditional on correct ε ; violations break those guarantees.

Challenge 7: Cross-Hash Synchronisation

Problem. The cryptographic cross-hashing protocol requires each integrator to include in its commitment hash the hashes received from all adjacent integrators: $H_j = \text{Hash}(\text{bids}_j, \text{alloc}_j, \text{payments}_j, \{H_k : k \in \text{adj}(j)\})$. This creates a dependency chain across integrators.

Why it matters. All adjacent integrators must complete their epochs and exchange hashes before any can finalise commitment. If one integrator is slow or unresponsive, it blocks the chain. With m integrators in a chain, the worst-case latency is $O(m)$ sequential hash exchanges.

Tension. Real-time service markets require sub-second epochs. If the hash chain introduces seconds of synchronisation delay, it may be incompatible with the latency budget. Asynchronous

alternatives (each integrator commits independently, FCA checks consistency post-hoc) sacrifice the tamper-evidence that the chain provides.

Severity: MEDIUM. The cross-hash is one of two audit options (the other being FCA-based). If cross-hashing is impractical, the system falls back to FCA, which introduces a trusted third party.

Challenge 8: Dual-Role Trust Concentration

Problem. The integrator simultaneously: 1. **Encapsulates** the sub-DAG (infrastructure role: managing physical resources). 2. **Operates** the mechanism (market role: running the auction, computing payments).

These roles create a concentration of trust. The integrator knows the bids (as mechanism operator), controls the resources (as infrastructure manager), and determines the mapping between abstract slices and physical resources (as encapsulator). No other entity in the architecture has comparable information or control.

Why it matters. Every credibility mechanism in Papers 2-3 addresses *execution* deviations (the integrator running a different mechanism than announced). But the dual role enables *design* deviations that are harder to detect: the integrator can design slice definitions that systematically favour certain agents, or set capacity bounds that create artificial scarcity.

Tension. Separating the roles (one entity manages infrastructure, another runs the auction) would reduce trust concentration but introduce a new coordination problem and potentially break the polymatroidal abstraction (the mechanism operator needs to know the true capacity, which only the infrastructure manager knows).

Severity: CRITICAL (under payment-based revenue); RESOLVED (under per-unit-fee). This is the fundamental trust bottleneck of the architecture. The theory addresses execution credibility thoroughly but largely assumes the encapsulation (sub-DAG to slice mapping) is performed honestly. Paper 3's Remark 2 shows that domain separation (per-unit-fee revenue) resolves this: the integrator's profit is increasing in allocation volume, making truthful capacity reporting incentive-compatible. Under payment-based revenue, encapsulation credibility remains open.

Challenge 9: Death Spiral Feedback Loop

Problem. Proposition 2 establishes that when an integrator's credibility degrades, high-value agents exit first (assortative sorting). This reduces revenue, which may reduce the integrator's investment in compliance, further degrading credibility. The loop is self-reinforcing.

Why it matters. The Governance Interface publishes q_j , and strategic agents condition their participation on it. A single credibility event can trigger a cascade that is difficult to reverse, even if the integrator subsequently behaves honestly.

Tension. The theory bounds the welfare loss (Prop. 2) but does not provide a mechanism to halt the cascade once it begins. The Market Segmentation Controller (governance-architecture.md, Component 2.6) can detect the spiral, but intervention options (routing agents to alternative integrators, subsidising the affected integrator) require mechanisms not specified in the theory.

Severity: MEDIUM-HIGH. The death spiral is a market-level failure that affects agents, not just the integrator. Recovery requires external intervention (governance) and time (rebuilding reputation with elevated prior).

Challenge 10: Epoch Boundary Synchronisation

Problem. The seven-phase epoch (Section 3) requires strict sequencing: slice construction before mechanism execution, mechanism execution before commitment, commitment before audit, audit before governance update. Across multiple integrators, epochs must be synchronised (or at least their boundaries aligned) for cross-market audit to be meaningful.

Why it matters. If integrator j runs epoch t while integrator k is still in epoch $t - 1$, the cross-hash chain cannot be computed, and the FCA cannot perform simultaneous cross-market verification. Asynchronous epochs create windows where coalition deviations are undetectable.

Tension. Different service domains may have different natural epoch lengths (a latency-sensitive edge service may need 100ms epochs; a batch cloud service may use 10s epochs). Forcing all domains to the same epoch length is wasteful for some and insufficient for others.

Severity: MEDIUM. Epoch synchronisation is an engineering challenge with known solutions (barrier synchronisation, epoch alignment protocols), but it introduces latency overhead that compounds with the number of integrators.

Challenge 11: Recursive Credibility of Audit Infrastructure

Problem. The FCA audits integrators. But who audits the FCA? If the FCA is a trusted third party, its own credibility is assumed, not verified. If it is a consortium of domain operators, those operators may have incentives to collude. If it is a smart contract, its correctness depends on the oracle that feeds it market data.

Why it matters. The entire governance stack terminates at the FCA. If the FCA is compromised, all detection, penalty, and certification mechanisms fail simultaneously. This is not a gradual degradation; it is a single point of catastrophic failure.

Tension. Making the FCA itself auditable (e.g., via multi-party computation among domain operators) adds latency and complexity. Using blockchain anchoring provides tamper-evidence but requires consensus mechanisms that may be too slow for real-time markets. Regulatory backing provides authority but not technical verifiability.

Severity: CRITICAL. This is the recursive credibility problem flagged in all three papers. No purely technical solution exists within the current framework; external commitment (regulation, reputation, multi-party computation) is required.

6. Challenge Summary

#	Challenge	Severity	Subsystem(s)	Theoretical Basis	Mitigation Status
1	Max-flow recomputation	HIGH	Sub-DAG, Slice Constr.	R1.1, R1.5	Incremental max-flow algorithms; epoch-based updates
2	GS validation	HIGH	Slice Constr., Mech. Exec.	R1.2, Lemma 1	Slice design constraints; runtime checks

#	Challenge	Severity	Subsystem(s)	Theoretical Basis	Mitigation Status
3	Transcript integrity	CRITICAL	Mech. Exec., Commit & Audit	R2.1, R2.5	Broadcast mode; hash chains; TEE
4	Deposit sizing	HIGH	Commit & Audit, Revenue	R3.3, R3.9	Historical estimation; conservative bounds
5	State-dependent penalties	HIGH	Commit & Audit, Revenue	R3.4, Thm. 2(ii)	State-space coarsening; Bayesian estimation
6	Audit signal calibration	MEDIUM-HIGH	Governance	R3.5, Prop. 1	Adaptive ϵ estimation; calibration tests
7	Cross-hash synchronisation	MEDIUM	Commit & Audit	R3.2, Cor. 1	Async fallback to FCA; epoch alignment
8	Dual-role trust concentration	CRITICAL / RE-SOLVED	All	R2.1, Thm. 1, Rem. 2	Per-unit-fee resolves it (Rem. 2); payment-based remains open
9	Death spiral feedback	MEDIUM-HIGH	Governance, Revenue	R3.8, Prop. 2	External intervention; gradual recovery
10	Epoch synchronisation	MEDIUM	All	R3.2, Cor. 1	Barrier protocols; flexible epoch lengths
11	Recursive credibility of FCA	CRITICAL	Commit & Audit, Governance	All papers	MPC; blockchain anchoring; regulation

Three challenges are rated CRITICAL: - **Transcript integrity** (Challenge 3): single point of failure for the entire governance stack. - **Dual-role trust concentration** (Challenge 8): the integrator is trusted more than the theory can verify. - **Recursive credibility** (Challenge 11): the audit infrastructure itself needs credibility guarantees.

These three are interconnected: if transcript integrity fails, audit is useless; if the dual role is exploited at the design level, transcripts look clean; if the FCA is compromised, no one detects any of it. The robustness of the integrator architecture depends on addressing all three simultaneously.

7. Key Architectural Insight

The integrator is the component where theory meets implementation most tensely. The trilogy's theoretical results provide strong guarantees (DSIC, coalition detection, learning deterrence, dynamic certification) but all of them assume the integrator faithfully performs the encapsulation step (sub-DAG to polymatroidal slice). This assumption is:

1. **Unverifiable from outside:** No external entity can independently verify that the integrator's reported max-flow matches the true physical capacity of its sub-DAG.

2. **Incentive-incompatible under payment-based revenue:** VCG revenue is non-monotone in supply; under-reporting capacity creates artificial scarcity that increases agent externalities and thus VCG payments.
3. **Undetectable:** The mechanism transcript is consistent with honest execution over the reported (under-reported) polymatroid, so no execution-level audit detects the misreport.

Resolution (Remark 2 in Paper 3). Domain separation (per-unit-fee revenue) resolves the encapsulation credibility gap. Under per-unit-fee revenue $\phi \cdot |\{i : x_i^* > 0\}|$ with ϕ exceeding marginal provision cost, the integrator's profit is increasing in allocation volume. Since a larger polymatroid serves weakly more agents, truthful capacity reporting maximises profit. Over-reporting is detectable via SLA violations. This extends Paper 2's domain separation result: per-unit-fee provides both execution credibility (Paper 2) and encapsulation credibility (Paper 3, Remark 2).

What remains open. For payment-based revenue with commitment devices, encapsulation credibility is not addressed by any formal result in the trilogy. Possible approaches include: - **Capacity attestation:** Trusted hardware (TEE) or zero-knowledge proofs that the reported max-flow matches the actual sub-DAG. - **Competitive pressure:** Multiple integrators offering overlapping slice types, enabling agents to cross-check capacity claims. - **Empirical monitoring:** Statistical tests comparing advertised capacity to actual service delivery over time, feeding into the reputation system.

None of these is sufficient alone; a layered approach (mirroring the credibility stack for mechanism execution) is likely needed.